

Curs 4: Circuite Decodificatoare (DCD) Circuite Demultiplexoare (DMUX)

Cuprins:

- **Prezentarea DCD și DMUX: tabel de adevăr, structură internă, simboluri, variante constructive;**
- **Circuite DCD și DMUX realizate în structuri integrate;**
- **Extinderea DMUX (mărirea numărului de ieșiri);**
- **Utilizarea DCD în implementarea funcțiilor binare;**

1. Decodificatorul binar (DCD)

◆ Definiție

Decodificatorul binar este un circuit combinațional prevăzut cu n intrări și 2^n ieșiri. De regulă, intrările DCD sunt denumite *intrări de selecție* sau *intrări de adrese*.

Din punct de vedere funcțional, circuitul are proprietatea de a recunoaște o combinație binară aplicată pe intrările de selecție și de a semnaliza acest eveniment prin activarea unei singure ieșiri.

Numărul de combinații binare distincte (coduri de intrare) recunoscute de DCD este dependent de numărul intrărilor de selecție iar numărul combinațiilor ce pot fi semnalizate este dependent de numărul de ieșiri disponibile.

Între numărul intrărilor de selecție și numărul maxim de ieșiri ale unui DCD există relațiile:

$$nr_ieșiri = 2^{nr_selecții}$$

$$nr_selecții = \log_2(nr_ieșiri)$$

În cazul unui DCD cu 8 intrări, se obține $nr_selecții = \log_2(8) = 3$. Cele trei selecții sunt notate cu $S_2 S_1 S_0$.

Din punct de vedere constructiv, decodificatoarele pot avea: a) ieșiri active pe zero logic; b) ieșiri active pe unu logic;

◆ Notarea intrărilor de selecție

Modalitățile acceptate de notarea a intrărilor de selecție vor fi prezentate pentru cazul unui DCD cu 16 ieșiri, caz în care, conform relațiilor din problema anterioară, deducem că avem 4 intrări de selecție. Acestea pot fi notate folosind una din variantele de mai jos:

- a) **DCBA**, unde **A** este intrarea de selecție cu ponderea cea mai mică;
- b) **S₃S₂S₁S₀**, unde **S₀** este intrarea de selecție cu ponderea cea mai mică;
- c) **A₃A₂A₁A₀**, unde **A₀** este intrarea de selecție cu ponderea cea mai mică;
- d) **8421**, unde **1** este intrarea de selecție cu ponderea cea mai mică;

◆ DCD cu ieșiri active pe zero logic

În figura 1 se prezintă simbolul și tabelul de adevăr pentru un decodor cu 8 ieșiri.

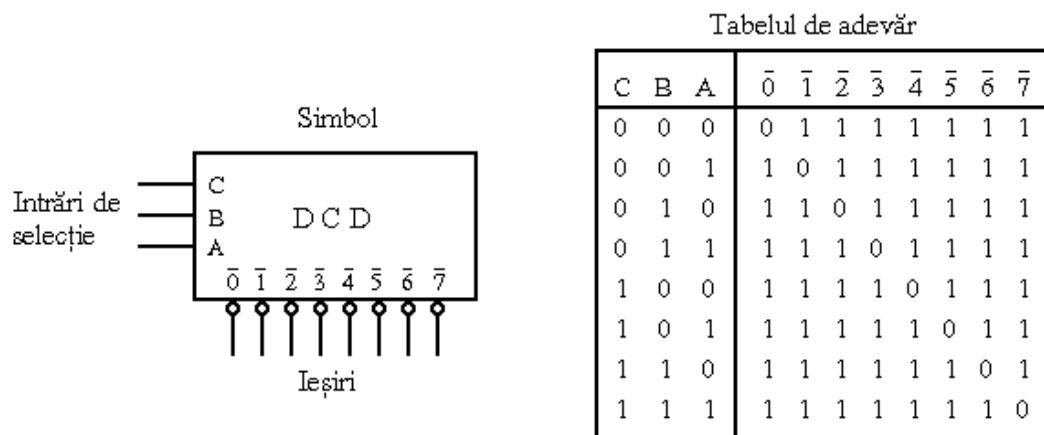


Fig. 1. DCD cu ieșiri active pe zero logic : simbol, structură internă

Observații:

- *prin convenție internațională*, ieșirile active pe zero logic sunt marcate printr-un mic cerculeț în simbolul logic al circuitului;
- *prin convenție internațională*, notarea ieșirilor active pe zero logic se face cu bară deasupra (exemplu, $\bar{4}$);
- activarea unei ieșiri înseamnă trecerea ei în zero logic;
- circuitul are asignată câte o ieșire pentru fiecare cod binar, distinct, aplicat pe intrările de selecție;
- în orice situație, **la un DCD găsim doar o singură ieșire activă**;

Structura internă a unui DCD se obține foarte ușor, plecând de la tabelul de adevăr și aplicând o metodă de sinteză a funcțiilor binare cu ajutorul rețelelor de porți logice. Etapele sintezei sunt ilustrate în figura de mai jos.

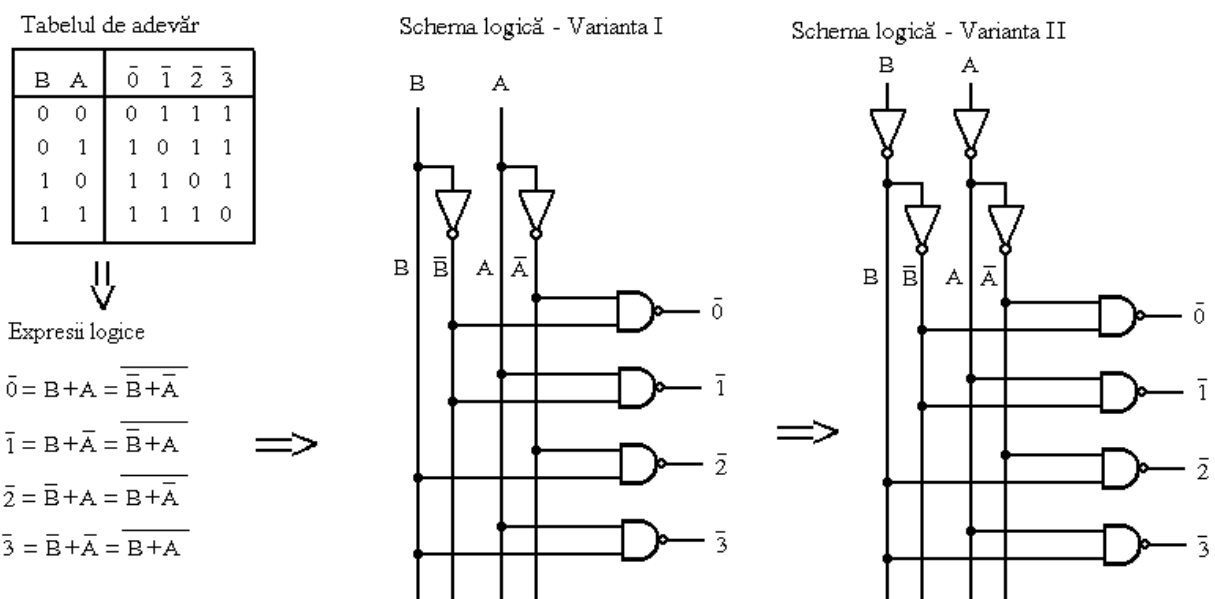


Fig. 2. Obținerea schemei interne a unui DCD cu ieșiri active pe zero logic

Obs:

- Cele două scheme din figura 2 sunt echivalente din punct de vedere logic dar, din punct de vedere al comportamentului electric, mai precis din punct de vedere al încărcării intrărilor de selecție, varianta II este mai avantajoasă.
- Pentru varianta I, fiecare semnal aplicat pe o intrare de selecție trebuie să comande un inversor și 2 porți NAND, deci factorul de încărcare al intrării este egal cu 3 sarcini logice. Factorul de încărcare crește și mai mult pentru decodificatoarele care au mai multe ieșiri deoarece apar mai multe porți NAND.
- Pentru varianta II, fiecare semnal aplicat pe o intrare de selecție trebuie să comande doar un singur inversor, ceea ce înseamnă că factorul de încărcare al intrării este egal cu 1 și nu depinde de numărul de ieșiri ale decodicatorului. Din acest motiv, varianta II, deși are un număr dublu de inversoare față de varianta I, este folosită în circuitele realizate integrat.

◆DCD cu ieșiri active pe unu logic

În figura 2 se prezintă simbolul și tabelul de adevăr pentru un decodor cu 4 ieșiri active pe unu logic

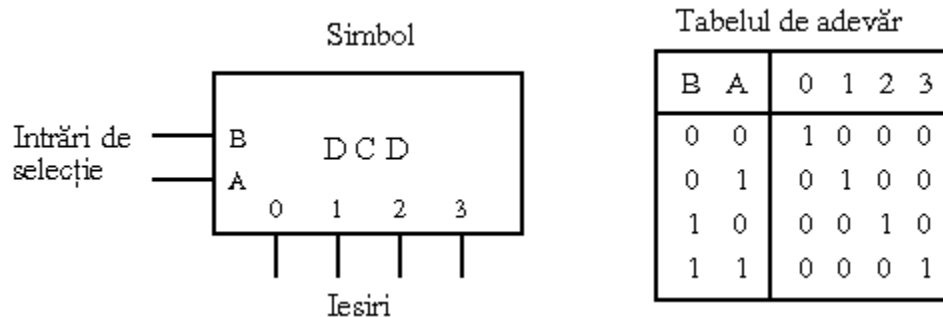


Fig. 3. DCD cu ieșiri active pe unu logic : simbol, tabel de adevăr

Observații:

- *prin convenție internațională*, ieșirile active pe unu logic nu mai au cerculeț în simbolul logic al circuitului;
- *prin convenție internațională*, notarea ieșirilor active pe unu logic se face fără bară deasupra notației;
- activarea unei ieșiri înseamnă trecerea ei în unu logic;
- circuitul are asignată câte o ieșire pentru fiecare cod binar, distinct, aplicat pe intrările de selecție;
- în orice situație, **la un DCD găsim doar o singură ieșire activă;**

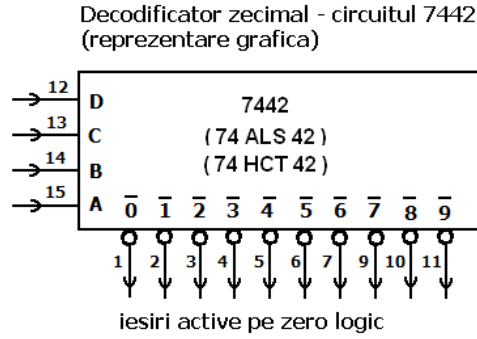
◆Exemplu: circuitul 74LS42 – decodificator zecimal

Circuitul 7442 este un decodificator zecimal cu ieșiri active pe zero logic, realizat în diverse tehnologii. Cele mai importante implementări ale acestui circuit sunt:

- 74 ALS 42 - **A**dvanced **L**ow Power **S**chottky, tehnologie bipolară de viteză mare și consum redus;
- 74 HC 42 – **H**igh Speed **C**MOS, tehnologie unipolară de mare viteză, compatibilă ca intrări cu circuitele logice TTL dar incompatibilă ca nivele logice;
- 74 HCT 42 – **H**igh Speed **C**MOS, tehnologie unipolară compatibilă la intrări și la nivele cu circuitele logice TTL;

Simbolul și tabelul de adevăr sunt prezentate în figura de mai jos.

Tabelul de adevăr pentru circuitul 7442



D	C	B	A	$\bar{0}$	$\bar{1}$	$\bar{2}$	$\bar{3}$	$\bar{4}$	$\bar{5}$	$\bar{6}$	$\bar{7}$	$\bar{8}$	$\bar{9}$
0	0	0	0	0	1	1	1	1	1	1	1	1	1
0	0	0	1	1	0	1	1	1	1	1	1	1	1
0	0	1	0	1	1	0	1	1	1	1	1	1	1
0	0	1	1	1	1	1	0	1	1	1	1	1	1
0	1	0	0	1	1	1	1	0	1	1	1	1	1
0	1	0	1	1	1	1	1	0	1	1	1	1	1
0	1	1	0	1	1	1	1	1	0	1	1	1	1
0	1	1	1	1	1	1	1	1	1	0	1	1	1
1	0	0	0	1	1	1	1	1	1	1	0	1	1
1	0	0	1	1	1	1	1	1	1	1	1	0	1
1	0	1	0	1	1	1	1	1	1	1	1	1	0
1	0	1	1	1	1	1	1	1	1	1	1	1	1
1	1	0	0	1	1	1	1	1	1	1	1	1	1
1	1	0	1	1	1	1	1	1	1	1	1	1	1
1	1	1	0	1	1	1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1	1	1	1	1	1	1

Fig. 4. Circuitul 74LS42 decodificator zecimal cu ieșiri active pe zero logic

Observații:

- Din tabelul de adevăr anterior se constată că doar primele 10 coduri binare sunt recunoscute și semnalizate prin activarea unei ieșiri.
- Pentru coduri binare mai mari decât 9 (zona gri a tabelului de adevăr) circuitul nu mai activează nicio ieșire.

◆ Exemplu: Implementarea unui DCD cu 8 ieșiri folosind circuitul 74LS42

- Din analiza tabelului de adevăr pentru circuitul 7442, constatăm că are inclus în el partea ce corespunde unui DCD cu 8 ieșiri (vezi zonele încercuite din figurile a și b).

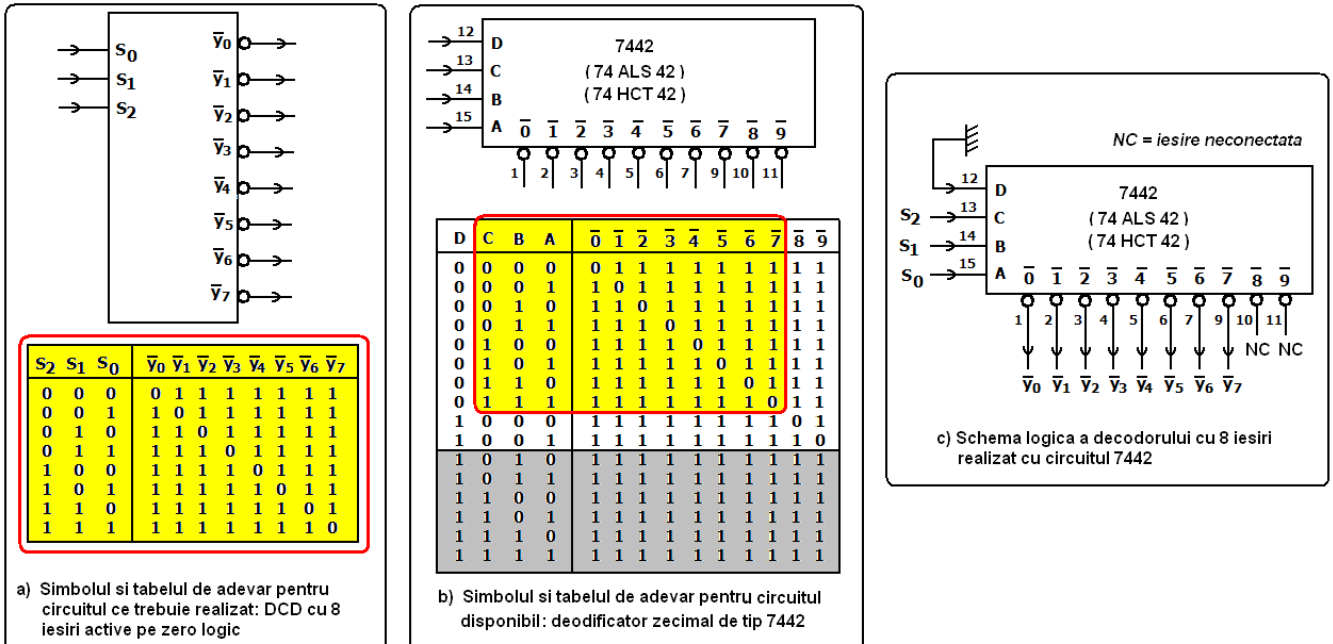


Fig.5. Utilizarea circuitului 74LS42 ca decodificator cu 8 ieșiri active pe zero logic

- Pentru a obține decodorul cu 8 ieșiri, trebuie să forțăm circuitul 7442 să opereze doar în zona încercuita a tabelului din figura b. Pentru aceasta este necesar ca intrarea D să fie legată în

permanență în zero logic, deci trebuie conectată la masă. În această situație, ieșirile $\bar{8}$ și $\bar{9}$, nu se vor mai activa niciodată, motiv pentru care vor fi nefolosite (lăsate în aer).

- Modul de utilizare a selecțiilor și a ieșirilor circuitului 7442 pentru implementarea DCD-ului dorit, se observă în figura c.

2. Demultiplexorul (DMUX)

Acest circuit poate fi privit ca un DCD prevăzut cu o intrare suplimentară de validare a funcționării circuitului. În consecință un DMUX prezintă: n intrări de selecție, o intrare de validare și 2^n ieșiri.

Intrarea de validare, denumită de regulă **Enable**, controlează funcționarea DMUX. Pentru cazul unui DMUX cu intrarea de validare activă pe zero logic, așa cum este cazul celui prezentat în figura 2, putem avea următoarele situații:

- dacă intrarea de validare este activată, $\bar{E} = 0$, funcționarea demultiplexorului este identică cu a unui DCD (în funcție de codul aplicat pe intrările de selecție se activează doar o singură ieșire);
- dacă intrarea de validare este neactivată, $\bar{E} = 1$, toate ieșirile sunt forțate în starea lor inactivă, indiferent de codul binar aplicat pe intrările de selecție.

La o primă vedere, se poate spune că DMUX este un DCD mai flexibil deoarece prin intermediul intrării de validare se poate controla momentele de timp în care DMUX are voie să funcționeze.

Principala funcție a unui DMUX este aceea de a distribui informația digitală prezentă pe intrarea de validare $\bar{E}$, spre una din liniile de ieșire. Este evident că selectarea liniei de ieșire se face prin intermediul intrărilor de selecție. **Deci, din punct de vedere funcțional, un DMUX poate fi asemănat cu un comutator rotativ.**

Pentru a demonstra această ultimă proprietate a unui DMUX, ce nu este evidentă la prima vedere, să considerăm că pe intrările de selecție ale circuitului din figura 2, aplicăm combinația binară $BA=01$, ceea ce înseamnă că ieșirea aleasă este $\bar{1}$. În funcție de stare logică a intrării $\bar{E}$ pot exista două situații:

- dacă $\bar{E} = 0$, circuitul DMUX lucrează ca un DCD și, datorită codului aplicat intrărilor de selecție, se activează ieșirea $\bar{1}$ (trece în starea "0") în timp ce restul ieșirilor rămân inactive (starea logică "1");
- dacă $\bar{E} = 1$, DMUX nu este validat, deci toate ieșirile sunt forțate în starea inactivă (starea logică "1").

Din analiza celor două situații se observă că starea logică a ieșirii selectate, în cazul de față $\bar{1}$, este identică cu starea logică a intrării de validare $\bar{E}$. Cu alte cuvinte putem afirma că **informația digitală prezentă pe intrarea $\bar{E}$ este transmisă (direcționată) spre o ieșire indicată de codul aplicat intrărilor de selecție.**

◆ Exemplul 1: circuitul 74LS139 - DMUX 1:4 cu ieșiri active pe zero logic

Un DMUX cu o intrare și patru ieșiri active pe zero este disponibil pe piață sub codul 74LS139. Structura internă, tabelul de adevăr precum și simbolul folosit pentru acest circuit sunt prezentate în figura 6

Din tabelul de adevăr se observă:

- pentru $\bar{E} = 0$, circuitul este validat și lucrează ca un DCD binar (pentru fiecare cod binar aplicat pe selecții, circuitul răspunde prin activarea unei ieșiri);
- activarea unei ieșiri presupune trecerea ei în zero logic, de aici vine și denumirea de *circuit cu ieșiri active pe zero*;

pentru $\bar{E} = 1$, circuitul este nevalidat și provoacă trece în starea inactivă a tuturor ieșirilor fără a mai ține cont de informația de pe intrările de selecție.

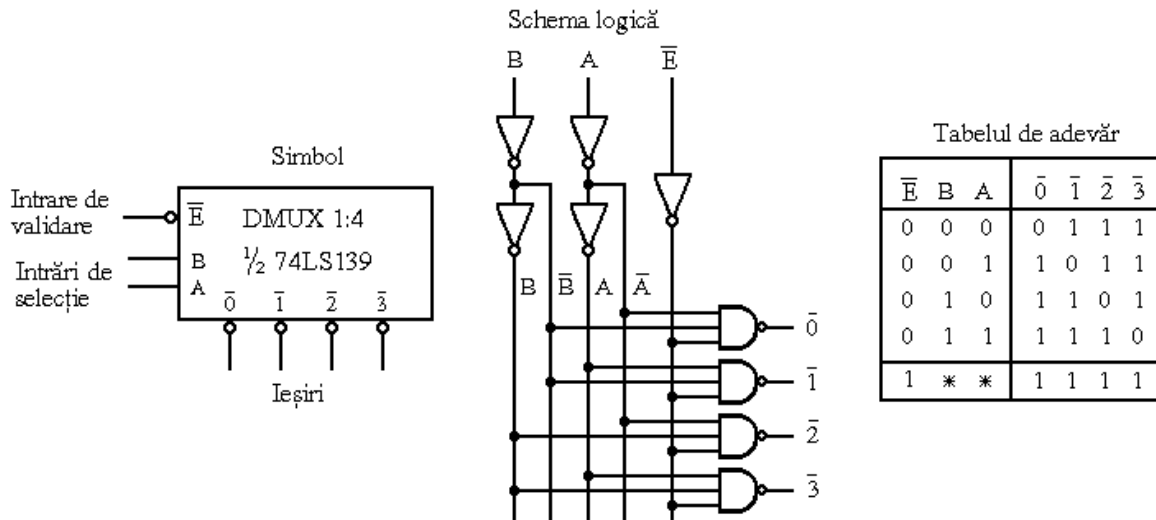


Fig.6. Schema logică, simbolul și tabelul de adevăr pentru DMUX 1:4 de tip 74LS139

◆ Exemplul 2: circuitul 74LS138 - DMUX 3:8 cu ieșiri active pe zero logic

Un circuit DMUX foarte des utilizat este circuitul 74LS138, care prezintă:

- 8 ieșiri de date active pe zero logic
- trei intrări de validare, două active pe zero logic iar a treia pe unu logic.
- activarea unei ieșiri pentru acest circuit se poate face numai dacă simultan sunt îndeplinite condițiile: $G1A=1$, $\overline{G2A}=0$ și $\overline{G2B}=0$.

Schema logică și tabelul de adevăr sunt prezentate în figura 7.

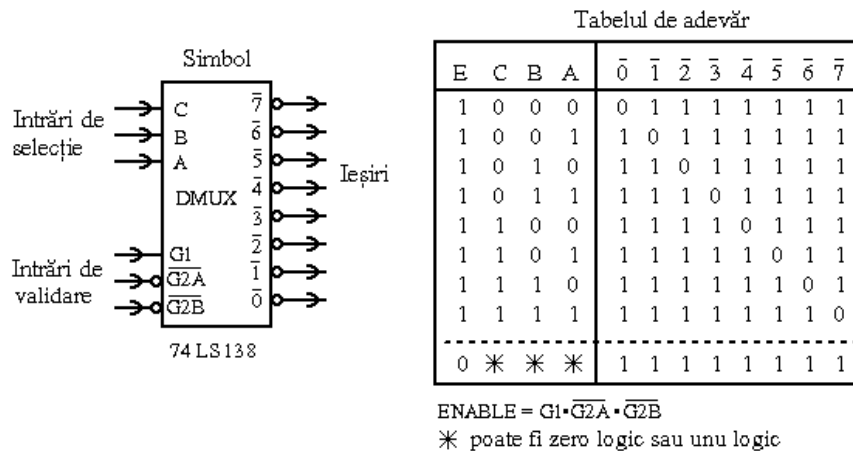


Fig.7. Schema logică și tabelul de adevăr pentru circuitul demultiplexor 3:8 de tip 74LS138

Din punct de vedere funcțional, circuitul 74LS138 poate fi echivalat cu un DMUX1:8 obișnuit la care se adaugă o logică suplimentară în scopul creșterii numărului de intrări de validare așa cum se prezintă în figura 8.

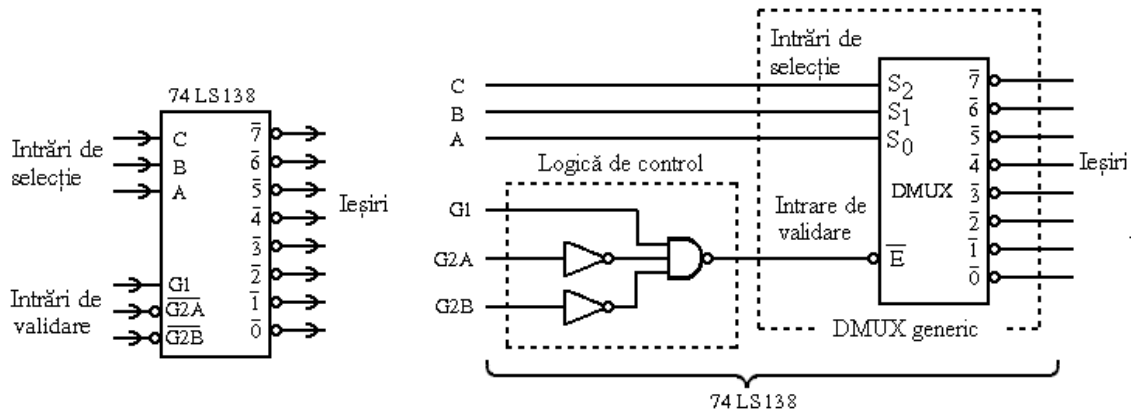


Fig.8. Echivalența logică a circuitului 74LS138

3. Extensia DMUX

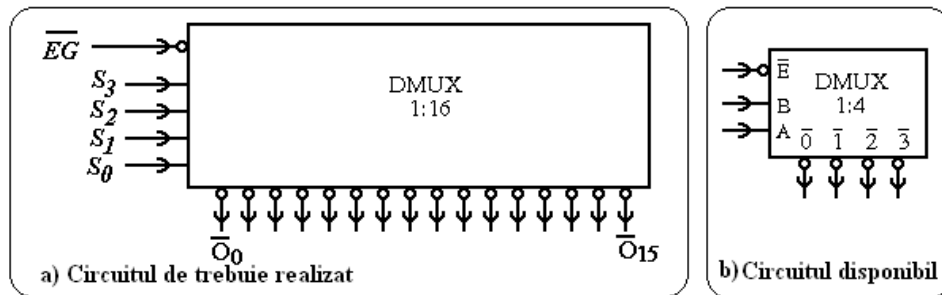
În unele aplicații practice apare necesitatea unor circuite DMUX cu un număr de ieșiri mai mare decât al circuitelor disponibile în mod curent pe piață. Pentru astfel de situații trebuie să folosim mai multe circuite DMUX disponibile, cu un număr mai mic de intrări, conectate între ele astfel încât să se comporte ca și cum ar fi unul singur.

Exemplu: Realizarea DMUX 1:16 folosind circuite de tip DMUX 1:4.

Pentru extensia numărului de ieșiri este necesar să parcurgem următoarele etape:

- Se desenează alăturat circuitul ce trebuie realizat și circuitul disponibil.

Această etapă are drept scop punerea în evidență a tuturor semnalelor ce intervin în cele două circuite. Prin convenție semnalele caracteristice circuitului ce trebuie realizat se scriu în afara dreptunghiului, iar semnalele circuitului de lucru se trec în interiorul acestuia.



- Se calculează numărul de circuite necesare etajului inferior raportând numărul de ieșiri ale circuitului de realizat la numărul de ieșiri disponibile ale circuitului de lucru.

Pentru cazul de față, necesarul de circuite din nivelul inferior este de: $16 \div 4 = 4$ circuite DMUX 1:4. În figura c, etajul inferior este realizat cu circuitele U2÷U5. Dacă, spre exemplu, lucrăm cu circuite DMUX 1:8 numărul de circuite s-ar fi redus la jumătate.

- Se determină numărul de circuite din etajul imediat următor. De data aceasta se raportează numărul de circuite din etajul inferior la numărul de ieșiri ale circuitelor utilizate în acest etaj. Trebuie menționat că nu este obligatorie utilizarea aceluiași tip de circuit decât în componența aceluiași etaj. Această etapă se reia până când numărul de circuite dintr-un etaj este egal cu unu.

În cazul de față, pe nivelul inferior avem 4 circuite iar circuitul de lucru dispune tot de 4 ieșiri, ceea ce înseamnă că este suficient un singur DMUX 1:4 pe acest etaj. Deoarece am ajuns la un singur circuit pe etaj (vezi circuitul U1 din figura c), acesta va fi chiar ultimul etaj al schemei.

- d) Realizarea schemei logice, deci a conexiunilor între etaje, se face respectând următoarele reguli:
- se împarte cuvântul de cod al circuitului de realizat, într-un număr de câmpuri egal cu numărul de etaje,
 - fiecare câmp are un număr de biți egal cu numărul de selecții ale circuitelor utilizate în etajul respectiv;
 - partea cea mai semnificativă a cuvântului de selecție, mai sus divizat, se recomandă a se aplica etajului superior, iar partea cea mai puțin semnificativă etajului inferior;
 - la nivelul fiecărui etaj se fac conexiuni între toate intrările de selecție cu aceeași pondere;
 - se recomandă ca intrările de selecție aplicate unui etaj să fie conectate respectând ponderile;
 - ieșirile decodoarelor dintr-un etaj se conectează la intrările de validare ale etajului imediat inferior.

În cazul de față, cuvântul de selecție $S_3S_2S_1S_0$ s-a despărțit în două câmpuri: primul S_3S_2 s-a aplicat etajului superior (U1), iar ultimul S_1S_0 s-a aplicat etajului inferior (U2÷U5). La nivelul etajului superior respectarea ponderilor înseamnă conectarea intrării de selecție S_3 la intrarea B a circuitului U1, respectiv S_2 la A.

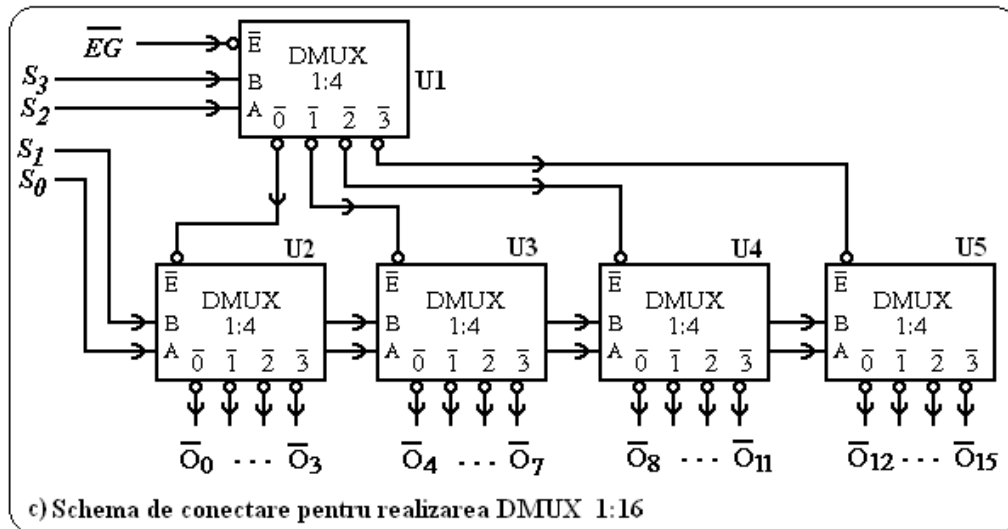


Fig.9. Realizarea unui DMUX 1:16 folosind circuite DMUX 1:4

La nivelul etajului inferior, respectarea ponderilor înseamnă conectarea intrării de selecție S_1 la intrarea B a circuitelor U2÷U5, respectiv S_0 la intrarea A a circuitelor U2÷U5.

Modul de conectare a celor 5 circuite DMUX 1:4 pentru a obține un circuit DMUX1:16 este prezentat în figura c. Din această figură se constată că intrarea de validare a circuitului U1 joacă rol de intrare de validare pentru circuitul mare cu 16 ieșiri, și este notată $\overline{EG}$.

Pentru codul de intrare $S_3S_2S_1S_0 = 1000$ schema lucrează astfel:

- pe nivelul primar de selecție, circuitul primește codul de selecție $BA = S_3S_2 = 10$, ceea ce are ca efect activarea ieșirii $\overline{2}$, care, la rândul său, va determina validarea circuitului U4 din nivelul inferior de selecție;
- pe nivelul inferior, circuitele U2÷U5 primesc codul de selecție $BA = S_1S_0 = 00$, ceea ce ar putea determina activarea ieșirilor $\overline{0}$ ale acestor circuite (ieșiri ce corespund notațiilor $\overline{O_0}$, $\overline{O_4}$, $\overline{O_8}$ respectiv $\overline{O_{12}}$). Deoarece numai U4 este validat, doar ieșirea $\overline{0}$ a acestuia va fi activată, ceea ce înseamnă că $\overline{O_8}$ este activă.

Din exemplul anterior se observă că schema lucrează corect, pentru codul de selecție $S_3S_2S_1S_0 = 1000$, se activează o singură ieșire, iar aceasta este $\overline{O_8}$.

4. Utilizarea DCD/DMUX în implementarea funcțiilor binare

Analizând schema logică a unui decodificator se observă că aceasta este organizată pe două nivele:

- un nivel de inversoare pentru calculul complementelor variabilelor de intrare;
- un nivel de NAND-uri (dacă ieșirile decodificatorului sunt active pe zero logic) sau un nivel de AND-uri (dacă ieșirile decodificatorului sunt active pe unu logic).

În ipoteza că pe intrările de selecție ale unui decodificator binar se aplică variabilele unei funcții binare, la ieșirile acestuia se vor regăsi fie mintermenii respectivei funcții (dacă DCD are ieșiri active pe unu logic) fie complementul mintermenilor (dacă DCD are ieșiri active pe zero logic).

Dacă variabilele funcției sunt conectate la intrările de selecție cu respectarea ponderilor, atunci numerotarea ieșirilor DCD-ului va fi identică cu numerotarea mintermenilor (m_0 este disponibil la ieșirea 0, m_1 este disponibil la ieșirea 1, ...).

Așadar, utilizarea DCD-ului este avantajoasă acolo unde este nevoie de implementarea mai multor funcții binare ce depind de aceleași variabile de intrare deoarece, mintermenii se calculează o singură dată și pot fi utilizați pentru fiecare funcție.

Regulile de conectare sunt următoarele:

- Se recomandă ca variabilele funcției binare să fie conectate la intrările de selecție ale DCD în ordinea ponderilor. Variabila funcției binare cu ponderea cea mai mică, x_0 , se conectată la intrarea de selecție cu ponderea cea mai mică, A, a circuitului. Conectarea variabilelor se poate face și în altă ordine, însă, în aceste cazuri, trebuie realizat un tabel de echivalență între combinația variabilelor funcției și combinația de selecție percepută efectiv de DCD.
- Dacă funcția logică prezintă mai puține valori de unu logic în tabelul său de adevăr se recomandă utilizarea unei porți OR sau a unei porți NAND după cum ieșirile DCD sunt active pe unu logic sau pe zero logic. Intrările porții se conectează la ieșirile DCD ce corespund mintermenilor pentru care funcția logică are valoarea de unu logic. Această situație este prezentată în figura 3.a) și respectiv 3. b).
- Dacă funcția logică prezintă mai puține valori de zero logic în tabelul său de adevăr atunci se recomandă utilizarea unei porți NOR sau a unei porți AND după cum ieșirile DCD sunt active pe unu logic sau pe zero logic. Intrările porții se conectează la ieșirile DCD ce corespund mintermenilor pentru care funcția logică are valoarea de zero logic. Această situație este prezentată în figura 3.4.c) și respectiv 3.4. d).
- Dacă numărul variabilelor de intrare ale funcției de implementat este mai mare decât numărul intrărilor de selecție, pot rezulta implementări complexe. De aceea, în aceste cazuri, apelarea la această metodă nu dă rezultatele cele mai bune.
- **Utilizarea circuitelor DCD se recomandă în situațiile în care este necesară implementarea mai multor funcții binare ce au aceleași variabile de intrare. În acest mod mintermenii sunt calculați o singură dată și sunt utilizați în comun pentru toate funcțiile ce trebuie implementate.**

Exemplul 1 : Fie o funcție binară F , dată prin tabelul de adevăr alăturat.

Se cere implementarea funcției în două cazuri:

- folosind DCD cu ieșiri active pe unu logic;
- folosind DCD cu ieșiri active pe zero logic.

x_2	x_1	x_0	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0

Rezolvare: Pentru rezolvarea acestei probleme vom scrie funcția F folosind cele două forme canonice, după care, acestea vor fi prelucrate până la punerea în evidență a mintermenilor sau a maxtermenilor negați.

Subpunctul a): varianta 1

Prima formă canonică (forma disjunctivă) a funcției F are următoarea expresie:

$$F = \left| \begin{array}{c} \bar{x}_2 x_1 \bar{x}_0 \\ x_2 \bar{x}_1 x_0 \end{array} \right| = \left| \begin{array}{c} m_2 \\ m_5 \end{array} \right| \quad (3.1)$$

Relația (3.1) conduce la o schemă de implementare a funcției F ca cea din figura 10.a)

Subpunctul a): varianta 2

A doua formă canonică (forma conjunctivă) a funcției F are următoarea expresie:

$$F = \left| \begin{array}{c} x_2 | x_2 | x_2 | \bar{x}_2 | \bar{x}_2 | \bar{x}_2 \\ x_1 | x_1 | \bar{x}_1 | x_1 | \bar{x}_1 | \bar{x}_1 \\ x_0 | \bar{x}_0 | \bar{x}_0 | x_0 | x_0 | \bar{x}_0 \end{array} \right| \quad (3.2)$$

Se observă că în relația (3.2) nu apar mintermenii funcției, deci în această formă relația nu este utilă pentru scopul propus. Evidențierea mintermenilor se face prin aplicarea în mod convenabil a teoremelor lui DeMorgan așa cum se arată mai jos:

$$F = \left| \begin{array}{c} x_2 | x_2 | x_2 | \bar{x}_2 | \bar{x}_2 | \bar{x}_2 \\ x_1 | x_1 | \bar{x}_1 | x_1 | \bar{x}_1 | \bar{x}_1 \\ x_0 | \bar{x}_0 | \bar{x}_0 | x_0 | x_0 | \bar{x}_0 \end{array} \right| = \left| \begin{array}{c} x_2 | x_2 | x_2 | \bar{x}_2 | \bar{x}_2 | \bar{x}_2 \\ x_1 | x_1 | \bar{x}_1 | x_1 | \bar{x}_1 | \bar{x}_1 \\ x_0 | \bar{x}_0 | \bar{x}_0 | x_0 | x_0 | \bar{x}_0 \end{array} \right| = \overline{\overline{M_0 M_1 M_3 M_4 M_6 M_7}} = \left| \begin{array}{c} \bar{M}_0 \\ \bar{M}_1 \\ M_3 \\ \bar{M}_4 \\ \bar{M}_6 \\ \bar{M}_7 \end{array} \right| \quad (3.3)$$

În relația (3.3) prin litera M_i sau notat maxtermenii ai funcției binare. Ținând cont că între mintermenul m_i și maxtermenul M_i există relația de legătură: $\bar{M}_i = m_i$ relația (3.3) se poate scrie și sub următoarea formă:

$$F = \left| \begin{array}{c} \bar{M}_0 \\ \bar{M}_1 \\ M_3 \\ \bar{M}_4 \\ \bar{M}_6 \\ \bar{M}_7 \end{array} \right| = \left| \begin{array}{c} m_0 \\ m_1 \\ m_3 \\ m_4 \\ m_6 \\ m_7 \end{array} \right| \quad (3.4)$$

Relația (3.4) conduce la o schemă de implementare a funcției F ca cea din figura 10.c)

Subpunctul b): varianta 1

Aplicând teoremele lui DeMorgan pentru relația (3.1) se obține:

$$F = \left| \begin{array}{c} \bar{x}_2 x_1 \bar{x}_0 \\ x_2 \bar{x}_1 x_0 \end{array} \right| = \left| \begin{array}{c} m_2 \\ m_5 \end{array} \right| = \overline{\overline{m_2 m_5}} = \overline{m_2 m_5} \quad (3.5)$$

Implementarea funcției după relația (3.5) este prezentată în figura 10.b).

Subpunctul b): varianta 2

Aplicând teoremele lui DeMorgan pentru relația (3.4) se obține:

$$F = \begin{matrix} m_0 \\ m_1 \\ m_3 \\ m_4 \\ m_6 \\ m_7 \end{matrix} = \overline{m_0} \overline{m_1} \overline{m_3} \overline{m_5} \overline{m_6} \overline{m_7} \quad (3.6)$$

Implementarea funcției F după relația (3.6) se prezintă în figura 10.d).

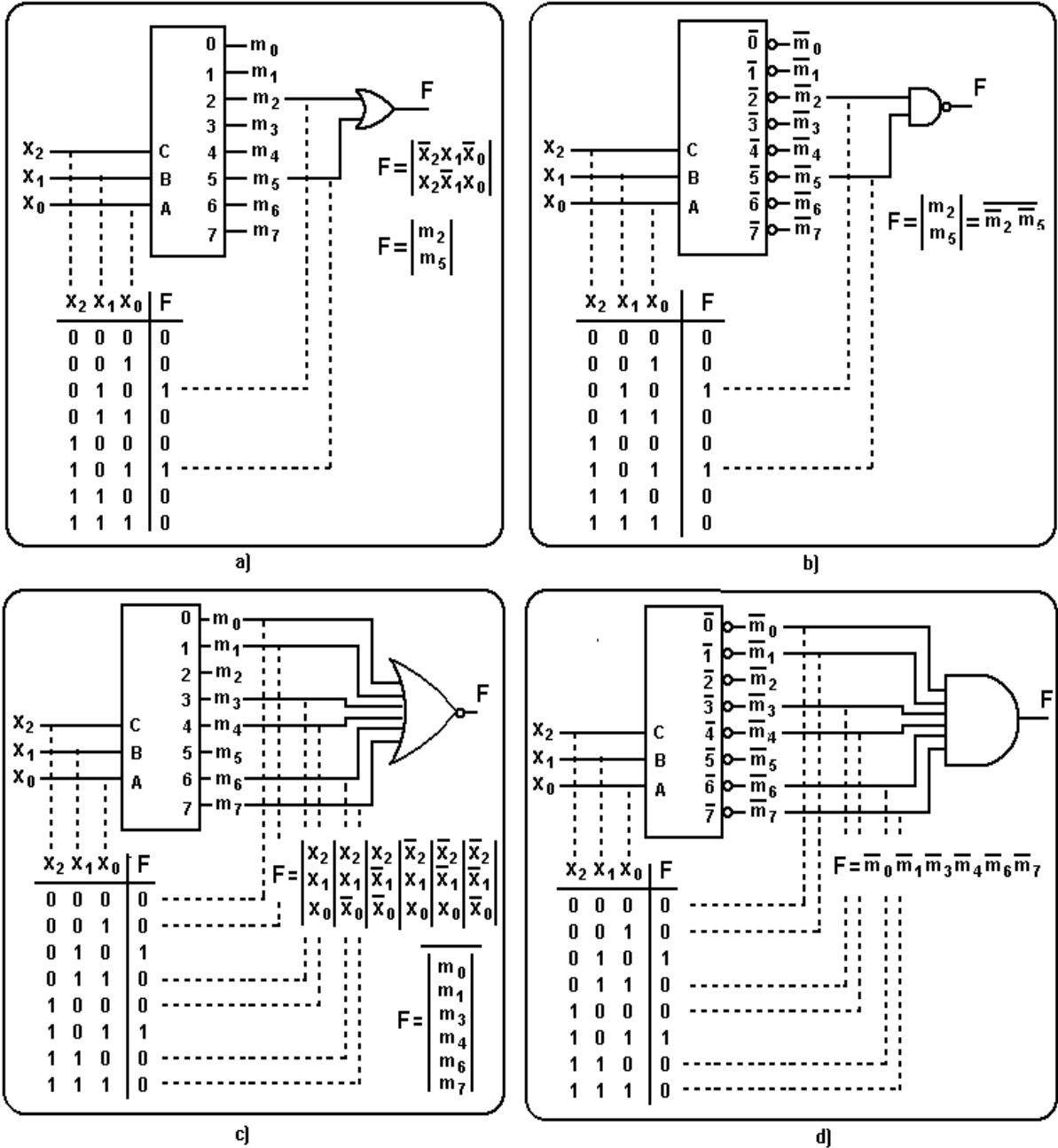


Fig.10. Implementarea funcției binare F ; a) și c) folosind DCD cu ieșiri active pe unu logic; b) și d) folosind DCD cu ieșiri active pe zero logic;

Avantajele utilizării DCD/DMUX în implementarea funcțiilor binare

Implementarea cu DCD a unei funcții booleene nu necesită operații de minimizare. La ieșirea DCD se obțin toți termenii canonici (în formă negată dacă ieșirile DCD sunt active pe zero logic, sau sub formă directă dacă ieșirile DCD sunt active pe unu logic).

Decodicatorul este foarte util și avantajos din punct de vedere economic în situațiile în care avem de implementat mai multe funcții binare ce depind de aceleași variabile de intrare. Avantajul vine din faptul că mintermenii sunt calculați o singură dată și folosiți de câte ori este nevoie.

Pe lângă DCD, pentru fiecare funcție binară mai avem nevoie de câte o poartă logică așa cum s-a arătat în exemplele anterioare.